



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO – ENERO 2026



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Recorridos Árboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3do-B
Alumnos participantes:	Curillo Pilamunga Diego Alexander
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar programas en C++ y Java que permitan implementar recorridos de árboles binarios para comprender el funcionamiento de las estructuras jerárquicas y los algoritmos recursivos.

Específicos:

- Analizar el funcionamiento de los árboles binarios y sus diferentes tipos de recorridos mediante la implementación de algoritmos recursivos.
- Estudiar las diferencias y similitudes entre la implementación de árboles binarios en los lenguajes C++ y Java.
- Desarrollar programas funcionales que permitan crear y recorrer árboles binarios utilizando estructuras dinámicas y programación orientada a objetos.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 4

No presenciales: 0

2.4 Descripción del trabajo realizado

El proyecto consiste en la implementación de recorridos de árboles binarios utilizando los lenguajes de programación C++ y Java. Se desarrollaron diferentes ejercicios enfocados en la creación de nodos, inserción de datos y recorridos de árboles mediante algoritmos recursivos.

2.5 Listado de equipos, materiales y recursos

Inteligencia artificial, TAC

Material bibliográfico, internet, IA, Laptop, C++, IDE Visual Studio Code.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒ Plataformas educativas

☒ Simuladores y laboratorios virtuales

☐ Aplicaciones educativas

☒ Recursos audiovisuales

☐ Gamificación

☐ Inteligencia Artificial

Otros (Especifique): _____



2.6 Actividades por desarrollar

- Clonar el repositorio.
- Ejecutar el código base.
- Agregar mínimo 5 nodos nuevos.
- Mostrar los cuatro recorridos.
- Modificar el caso de aplicación al proyecto final.
- Subir evidencias al repositorio GitHub del grupo.
- Captura de ejecución en consola.
- Código fuente comentado.
- README del grupo.
- Explicación del caso real.
- Link del repositorio GitHub.

2.7 Resultados obtenidos

Recorrido Árbol

Repositorio del trabajo:

<https://github.com/DiegO2255/Tarea2RecorridoArbolesUTA/tree/main>

Tecnologías utilizadas

- Lenguaje de programación: C++ y JAVA
- Entorno de desarrollo: Visual Studio Code
- Compilador: g++ y java
- Control de versiones: Git
- Plataforma: GitHub

Implementación en C++

Archivo: Ejercicio.cpp

Descripción

Este programa implementa la creación y recorrido de un árbol binario en C++.

Se utilizan nodos enlazados dinámicamente para almacenar los datos del árbol y ejecutar recorridos recursivos.

Funcionalidades

Creación de nodos del árbol.

- Inserción de elementos.
- Recorrido:
 - Preorden
 - Inorden
 - Postorden



Capturas de Ejecución del Programa

1. Insertar 5 Nodos

```
--- MENU ARBOL BINARIO ---
1. Insertar nodo manualmente
2. Mostrar Preorden
3. Mostrar Inorden
4. Mostrar Postorden
5. Mostrar BFS (Amplitud)
0. Salir
Seleccione: 1
Valor: 35

--- MENU ARBOL BINARIO ---
1. Insertar nodo manualmente
2. Mostrar Preorden
3. Mostrar Inorden
4. Mostrar Postorden
5. Mostrar BFS (Amplitud)
0. Salir
Seleccione: 1
Valor: 40

--- MENU ARBOL BINARIO ---
1. Insertar nodo manualmente
2. Mostrar Preorden
3. Mostrar Inorden
4. Mostrar Postorden
5. Mostrar BFS (Amplitud)
0. Salir
Seleccione: 1
Valor: 45

--- MENU ARBOL BINARIO ---
1. Insertar nodo manualmente
2. Mostrar Preorden
3. Mostrar Inorden
4. Mostrar Postorden
5. Mostrar BFS (Amplitud)
0. Salir
Seleccione: 1
Valor: 50

--- MENU ARBOL BINARIO ---
1. Insertar nodo manualmente
2. Mostrar Preorden
3. Mostrar Inorden
4. Mostrar Postorden
5. Mostrar BFS (Amplitud)
0. Salir
Seleccione: 1
Valor: 55
```

2. Preorden

```
--- MENU ARBOL BINARIO ---
1. Insertar nodo manualmente
2. Mostrar Preorden
3. Mostrar Inorden
4. Mostrar Postorden
5. Mostrar BFS (Amplitud)
0. Salir
Seleccione: 2
Preorden: 10 5 2 1 3 7 15 12 20 18 25 30 35 40 45 50 55
```

3. Inorden

```
--- MENU ARBOL BINARIO ---
1. Insertar nodo manualmente
2. Mostrar Preorden
3. Mostrar Inorden
4. Mostrar Postorden
5. Mostrar BFS (Amplitud)
0. Salir
Seleccione: 3
Inorden: 1 2 3 5 7 10 12 15 18 20 25 30 35 40 45 50 55
```

4. Postorden

```
--- MENU ARBOL BINARIO ---
1. Insertar nodo manualmente
2. Mostrar Preorden
3. Mostrar Inorden
4. Mostrar Postorden
5. Mostrar BFS (Amplitud)
0. Salir
Seleccione: 4
Postorden: 1 3 2 7 5 12 18 55 50 45 40 35 30 25 20 15 10
```



5. BFS

```
--- MENU ARBOL BINARIO ---
1. Insertar nodo manualmente
2. Mostrar Preorden
3. Mostrar Inorden
4. Mostrar Postorden
5. Mostrar BFS (Amplitud)
0. Salir
Seleccione: 5
BFS: 10 5 15 2 7 12 20 1 3 18 25 30 35 40 45 50 55
```

Archivo: Ejercicio5.cpp

Descripción

Este ejercicio implementa operaciones adicionales sobre árboles binarios en C++, permitiendo recorrer y visualizar los datos almacenados en la estructura.

Funcionalidades

- Construcción del árbol.
- Inserción de nodos.
- Recorridos del árbol.
- Visualización de resultados en consola.

Capturas de Ejecución del Programa

1. Preorden

```
--- MENU PROYECTO FINAL (ARBOLES) ---
1. Ver estructura del menu (Preorden)
2. Ejecutar procesamiento de modulos (Postorden)
3. Ver mapa del sitio por niveles (BFS)
0. Salir
Opcion: 1

ESTRUCTURA HIERARQUICA:
-> Sistema Web
-> Usuarios
-> Registrar
-> Buscar
-> Inventario
-> Productos
-> Reportes
```

2. Postorden

```
--- MENU PROYECTO FINAL (ARBOLES) ---
1. Ver estructura del menu (Preorden)
2. Ejecutar procesamiento de modulos (Postorden)
3. Ver mapa del sitio por niveles (BFS)
0. Salir
Opcion: 2

LOG DE PROCESAMIENTO:
[OK] Modulo 'Registrar' procesado.
[OK] Modulo 'Buscar' procesado.
[OK] Modulo 'Usuarios' procesado.
[OK] Modulo 'Productos' procesado.
[OK] Modulo 'Reportes' procesado.
[OK] Modulo 'Inventario' procesado.
[OK] Modulo 'Sistema Web' procesado.
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO – ENERO 2026



3. BFS

```
--- MENU PROYECTO FINAL (ARBOLES) ---
1. Ver estructura del menu (Preorden)
2. Ejecutar procesamiento de modulos (Postorden)
3. Ver mapa del sitio por niveles (BFS)
0. Salir
Opcion: 3

VISTA POR NIVELES:
[Sistema Web] [Usuarios] [Inventario] [Registrar] [Buscar] [Productos] [Reportes]
```

Implementación en Java

Archivo: Ejercicio.java

Descripción

Este programa desarrolla la implementación de árboles binarios utilizando Java. Se emplean clases y objetos para representar nodos y ejecutar los recorridos del árbol.

Funcionalidades

- Creación de nodos.
- Inserción de datos.
- Recorridos:
 - Preorden
 - Inorden
 - Postorden

Capturas de Ejecución del Programa

1. Insertar 5 nodos nuevos

```
1. Insertar | 2. Preorden | 3. Inorden | 4. Postorden | 5. BFS | 0. Salir
1
Valor: 35

1. Insertar | 2. Preorden | 3. Inorden | 4. Postorden | 5. BFS | 0. Salir
1
Valor: 40

1. Insertar | 2. Preorden | 3. Inorden | 4. Postorden | 5. BFS | 0. Salir
1
Valor: 45

1. Insertar | 2. Preorden | 3. Inorden | 4. Postorden | 5. BFS | 0. Salir
1
Valor: 50

1. Insertar | 2. Preorden | 3. Inorden | 4. Postorden | 5. BFS | 0. Salir
1
Valor: 55
```

2. Preorde – Inorden – Postorden – BFS

```
-----
1. Insertar | 2. Preorden | 3. Inorden | 4. Postorden | 5. BFS | 0. Salir
Elija una opcion: 2
PREORDEN: 10 5 2 1 3 7 15 12 20 18 22 25 30 35 40 45 50 55 60
-----

1. Insertar | 2. Preorden | 3. Inorden | 4. Postorden | 5. BFS | 0. Salir
Elija una opcion: 3
INORDEN: 1 2 3 5 7 10 12 15 18 20 22 25 30 35 40 45 50 55 60
-----

1. Insertar | 2. Preorden | 3. Inorden | 4. Postorden | 5. BFS | 0. Salir
Elija una opcion: 4
POSTORDEN: 1 3 2 7 5 12 18 60 55 50 45 40 35 30 25 22 20 15 10
-----

1. Insertar | 2. Preorden | 3. Inorden | 4. Postorden | 5. BFS | 0. Salir
Elija una opcion: 5
BFS (Por niveles): 10 5 15 2 7 12 20 1 3 18 22 25 30 35 40 45 50 55 60
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO – ENERO 2026



Archivo: Ejercicio5.java

Descripción

Este ejercicio implementa funcionalidades complementarias de recorridos de árboles binarios en Java, permitiendo visualizar el comportamiento de la estructura mediante la consola.

Funcionalidades

- Creación del árbol binario.
- Inserción de nodos.
- Recorridos del árbol.
- Impresión de resultados.

Capturas de Ejecución del Programa

1. Preorden

```
--- MENU SISTEMA WEB ---
1. Mostrar Jerarquia (Preorden)
2. Procesar Modulos (Postorden)
3. Mapa de Niveles (BFS)
0. Salir
Elija una opcion: 1
Modulo: Sistema Web
Modulo: Usuarios
Modulo: Registrar
Modulo: Buscar
Modulo: Inventario
Modulo: Productos
Modulo: Reportes
```

2. Postorden

```
--- MENU SISTEMA WEB ---
1. Mostrar Jerarquia (Preorden)
2. Procesar Modulos (Postorden)
3. Mapa de Niveles (BFS)
0. Salir
Elija una opcion: 2
Finalizando sub-modulo: Registrar
Finalizando sub-modulo: Buscar
Finalizando sub-modulo: Usuarios
Finalizando sub-modulo: Productos
Finalizando sub-modulo: Reportes
Finalizando sub-modulo: Inventario
Finalizando sub-modulo: Sistema Web
```

3. BFS

```
--- MENU SISTEMA WEB ---
1. Mostrar Jerarquia (Preorden)
2. Procesar Modulos (Postorden)
3. Mapa de Niveles (BFS)
0. Salir
Elija una opcion: 3
[Sistema Web] [Usuarios] [Inventario] [Registrar] [Buscar] [Productos] [Reportes]
```

Ejecución del programa



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO – ENERO 2026



Compilación cpp:

g++ src/main.cpp -o arbol

Ejecución cpp:

./arbol

Compilación java:

javac Ejercicio.java

Ejecución java:

java Ejercicio

Repositorio

<https://github.com/DiegO2255/Tarea2RecorridoArbolesUTA/tree/main>

2.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☒ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☐ Responsabilidad

2.9 Conclusiones

La implementación de recorridos de árboles binarios permitió comprender el funcionamiento de las estructuras jerárquicas y la importancia de la recursividad en el desarrollo de algoritmos. Además, se identificaron diferencias entre la implementación en C++ y Java, fortaleciendo los conocimientos de programación orientada a objetos y estructuras dinámicas.

2.10 Recomendaciones

Se recomienda continuar practicando con estructuras de datos más complejas para mejorar la lógica de programación y la comprensión de algoritmos recursivos. También es importante aplicar buenas prácticas de documentación y organización del código.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO – ENERO 2026



2.11 Anexos

The screenshot shows a GitHub repository named 'Tarea2RecorridoArbolesUTA' by user 'DiegO2255'. The repository is public and has 0 stars, 0 forks, and 0 watchers. The main branch is 'main' with 1 branch and 0 tags. The repository contains several files and folders, including 'CapturasCodigo', 'CapturasEjecucion', 'docs', 'exercises', 'moodle', 'src', '.gitignore', 'INDICACIONES_DOCENTE.md', 'LICENSE', and 'README.md'. The README is selected and shows the following content:

Implementación de Recorridos de Árboles en C++ y Java

Objetivo del proyecto

Implementar y analizar recorridos de árboles binarios utilizando C++ y Java para comprender el funcionamiento de estructuras jerárquicas y algoritmos recursivos.

Autor

Diego Alexander Curillo Pilamunga

Descripción

Este proyecto contiene la implementación de ejercicios relacionados con recorridos de árboles binarios utilizando los lenguajes C++ y Java. Se desarrollaron programas capaces de representar árboles y realizar diferentes tipos de recorridos para demostrar el funcionamiento de las estructuras de datos jerárquicas.

Repositorio del proyecto:
<https://github.com/DiegO2255/Tarea2RecorridoArbolesUTA/tree/main>

Implementación en C++

The right sidebar of the repository page shows the following information:

- About:** No description, website, or topics provided.
- Releases:** No releases published. [Create a new release](#)
- Packages:** No packages published. [Publish your first package](#)
- Contributors:** 1 contributor: DiegO2255
- Languages:** Java 55.0%, C++ 44.0%
- Suggested workflows:** Based on your tech stack. Suggested workflows include: Scala, Publish Java Package with Maven, and Java with Maven.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: AGOSTO – ENERO 2026



The screenshot displays an IDE with two open files. The top file, `Ejercicio5.java`, contains Java code for a tree traversal exercise. It includes a `preorden` method for pre-order traversal and a `postorden` method for post-order traversal. The `bfs` method implements a Breadth-First Search using a `LinkedList` as a queue. The bottom file, `Ejercicio5.cpp`, contains C++ code for the same exercise. It defines a `Modulo` struct with `nombre`, `izq`, and `der` attributes. It also implements `mostrarMenu`, `procesarInternos`, and `mostrarNiveles` functions. The `mostrarNiveles` function uses a queue to perform a BFS traversal of the tree.

```
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).
2 import java.util.LinkedList;
3 import java.util.Scanner;
4
5 public class Ejercicio5 {
6     public Ejercicio5() {
7     }
8
9     public static void preorden(Modulo var0) {
10         if (var0 != null) {
11             System.out.println("Modulo: " + var0.nombre);
12             preorden(var0.izq);
13             preorden(var0.der);
14         }
15     }
16
17     public static void postorden(Modulo var0) {
18         if (var0 != null) {
19             postorden(var0.izq);
20             postorden(var0.der);
21             System.out.println("Finalizando sub-modulo: " + var0.nombre);
22         }
23     }
24
25     public static void bfs(Modulo var0) {
26         if (var0 != null) {
27             LinkedList var1 = new LinkedList();
28             var1.add(var0);
29
30             while (!var1.isEmpty()) {
31                 Modulo var2 = (Modulo)var1.poll();
32                 System.out.print "[" + var2.nombre + " ] ";
33                 if (var2.izq != null) {
34                     var1.add(var2.izq);
35                 }
36                 if (var2.der != null) {
37                     var1.add(var2.der);
38                 }
39             }
40         }
41     }
42 }
```

```
1 #include <iostream>
2 #include <string>
3 #include <queue>
4
5 using namespace std;
6
7 struct Modulo {
8     string nombre;
9     Modulo *izq, *der;
10     Modulo(string n) : nombre(n), izq(nullptr), der(nullptr) {}
11 };
12
13 // Funciones de recorrido (Criterios de rúbrica)
14 void mostrarMenu(Modulo* m) { // Preorden
15     if (m) {
16         cout << " -> " << m->nombre << endl;
17         mostrarMenu(m->izq);
18         mostrarMenu(m->der);
19     }
20 }
21
22 void procesarInternos(Modulo* m) { // Postorden
23     if (m) {
24         procesarInternos(m->izq);
25         procesarInternos(m->der);
26         cout << "[OK] Modulo '" << m->nombre << "' procesado." << endl;
27     }
28 }
29
30 void mostrarNiveles(Modulo* raiz) { // BFS (Uso de cola)
31     if (!raiz) return;
32     queue<Modulo*> q;
33     q.push(raiz);
34     while (!q.empty()) {
35         Modulo* actual = q.front();
36         q.pop();
37         cout << "[" << actual->nombre << " ] ";
38     }
39 }
```